

TU Dortmund University — Department of Statistics

Case Studies – Summer Semester 2026

Project 2: Forecasting COVID-19

Time-Series:

A Machine Learning Approach

Lecturers: Prof. Dr. Matei Demetrescu, Dr. Paul Navas

Author: Danish Ahmad

Group members: Julia Wingenfeld, Ajeyaa Sinha Roy, Tom Schwaiger

June 16, 2026

Contents

1	Introduction	3
2	Problem and Data Description	4
2.1	Research Objectives and Scope	4
2.2	Data Sources and Features	4
3	Methods	5
3.1	Regression Trees (CART)	5
3.2	Multi-Layer Perceptrons (MLP)	6
3.3	Validation Strategy and Expanding-Window Estimation	6
3.4	Feature Importance	7
3.5	Software	7
4	Empirical Results	7
4.1	Part (a): Forecasting with Lags Only	7
4.2	Part (b): Forecasting with Lags and Covariates	8
4.3	Parts (c) and (d): Variable Importance	10
5	Conclusion	12
	Bibliography	14

1 Introduction

Daily COVID-19 case numbers are important because they can help governments and hospitals plan ahead. For example, if cases are expected to rise, hospitals may need to prepare more staff and capacity (Kucharski et al., 2020). However, forecasting COVID-19 cases is difficult. The reported numbers do not only depend on infections, but also on reporting delays, weekday effects, and changes in testing or reporting rules. This is visible in the data because the series has clear weekly ups and downs.

In Project 1, the focus was on linear time series models. In this project, we use machine learning methods instead. The advantage is that these methods can capture nonlinear patterns without assuming a fixed linear relationship between the variables (Györfi et al., 2002; Hastie et al., 2009). We compare two methods: a regression tree based on the CART algorithm (Breiman et al., 1984) and a multi-layer perceptron, which is a neural network (Rumelhart et al., 1986; Bishop, 2006).

The models are estimated in two versions. In Part (a), we only use the ten most recent lags of the log case series. In Part (b), we also include lagged covariates, such as temperature, age-group case numbers, and a Monday dummy. With these additional variables, the model has 100 input variables. In both parts, the goal is to predict the next day’s log case count. We tune the hyperparameters on a validation set and then evaluate the final forecasts on the test set. During the test period, the models are re-estimated step by step with an expanding window.

The results show that the neural network performs better than the regression tree. In Part (a), the neural network has an RMSFE of 0.5951, while the regression tree has an RMSFE of 0.7107. This difference is statistically significant according to the Diebold-Mariano test ($DM = 2.949$, $p = 0.003$). In Part (b), adding the covariates improves the neural network to an RMSFE of 0.5715. The regression tree becomes slightly worse, with an RMSFE of 0.7252. However, the improvement for the neural network is not statistically significant ($p = 0.3996$), and the change for the tree is also not statistically significant ($p = 0.333$).

The rest of the report is structured as follows. Section 2 describes the data and variables. Section 3 explains the methods and the forecasting setup. Section 4 presents the results, including forecast accuracy and variable importance. Section 5 concludes the report.

2 Problem and Data Description

2.1 Research Objectives and Scope

The aim of this project is to forecast the number of new COVID-19 cases in Germany for the next day. The target variable is the log-transformed number of daily cases:

$$y_t = \log(1 + \text{cases}_t). \quad (1)$$

We use the log transformation because the number of cases changes a lot over time, especially during large pandemic waves. The transformation makes the series less extreme and easier to model (Box and Cox, 1964). Therefore, the models do not predict the raw case numbers directly, but the transformed case series.

Both models use a maximum lag length of ten days. This means that the forecast for day $t + 1$ is based on information from day t back to day $t - 9$. This is important because only information that is already known at the time of forecasting should be used. Otherwise, future information would enter the model and the forecast evaluation would be biased (Tashman, 2000).

2.2 Data Sources and Features

The COVID-19 case data come from the national surveillance data of the Robert Koch Institute (RKI) (Robert Koch-Institut, 2026). The data contain daily reported cases for Germany. In this analysis, the dataset covers 2,193 daily observations from 31 March 2020 to 1 April 2026.

We included daily mean temperature as an additional variable. The temperature data is taken from the Open-Meteo historical weather API (Zippenfenig, 2026). Temperature is included because weather may be related to infection patterns and human behaviour (Merow and Urban, 2020).

Another important feature of the data is the weekly reporting pattern. Reported case numbers are usually lower on weekends because fewer reports are processed by local health offices. After the weekend, the numbers often increase again because delayed cases are added to the data. To capture this pattern, we include a Monday dummy variable called `start_of_week`. This variable is equal to 1 on Mondays and 0 on all other days.

The dataset also contains case numbers by age group. The age groups follow the RKI classification, from A00–A04 up to A80+ (Robert Koch-Institut, 2026). These age-group case numbers are also log-transformed before they are used in the models.

In Part (a), the models only use the 10 most recent lags of the log case series, namely

$y_t, \dots, y_{t-9}$. This gives 10 input variables. In Part (b), we add 10 lags of each covariate. These covariates are the age-group case numbers, temperature, and the Monday dummy. In total, the feature set in Part (b) contains 100 input variables.

3 Methods

The goal of both models is to predict the next day’s log case number. In general, the forecasting model can be written as

$$y_{t+1} = f(y_t, \dots, y_{t-9}, X_t, \dots, X_{t-9}) + \varepsilon_{t+1}, \quad (2)$$

where y_t is the log case number at time t , and X_t contains the additional covariates. The error term ε_{t+1} contains the part of the next day’s cases that cannot be predicted by the model.

All variables are used with lags only. This means that when we forecast day $t + 1$, we only use information that is already available up to day t . This is important because using future information would make the forecast unrealistically good.

3.1 Regression Trees (CART)

The first method is a regression tree. A regression tree splits the data into smaller groups by using simple decision rules (Breiman et al., 1984). For example, the tree may split the data depending on whether one lagged case value is above or below a certain value. At the end of the tree, each final group gives one prediction.

At each split, the tree chooses the variable and split point that reduce the squared prediction errors as much as possible:

$$\min_{j, s} \sum_{x_i \in R_1(j, s)} (y_i - \bar{y}_{R_1})^2 + \sum_{x_i \in R_2(j, s)} (y_i - \bar{y}_{R_2})^2. \quad (3)$$

Here, j is the selected variable, s is the split point, and R_1 and R_2 are the two groups created by the split.

If a tree is too deep, it can fit the training data too closely and perform worse on new data. Therefore, we tune two main hyperparameters: `max_depth`, which controls how deep the tree can become, and `min_samples_leaf`, which controls the minimum number of observations in each final leaf. The best values are chosen by comparing the validation-set RMSFE over a grid of possible values.

3.2 Multi-Layer Perceptrons (MLP)

The second method is a multi-layer perceptron. This is a type of neural network (Rumelhart et al., 1986; Bishop, 2006). An MLP combines the input variables through hidden layers and can learn nonlinear relationships between the predictors and the target variable.

Each hidden unit first calculates a weighted sum of the inputs. Then it applies an activation function. In this project, the ReLU activation function is used:

$$g(z) = \max(0, z). \quad (4)$$

This means that negative values are set to zero, while positive values stay unchanged.

The MLP is trained by adjusting its weights step by step to reduce the prediction error. The Adam optimiser is used to update the weights, and an ℓ_2 penalty is added to reduce overfitting (Hastie et al., 2009). Before training the MLP, all input variables are standardised using `StandardScaler`. This is necessary because neural networks can be sensitive to variables having different scales.

The network structure is chosen using the validation set. We compare different hidden-layer sizes, such as one hidden layer with 32 or 64 units, and two hidden layers such as 64 and 32 units or 128 and 64 units.

3.3 Validation Strategy and Expanding-Window Estimation

The data are split in chronological order because this is a time series. Random splitting is not used, because it could mix past and future observations. The first 80% of the observations are used as the training and validation block. The remaining 20% are used as the test set. This gives 1,746 observations for training and validation, and 437 observations for testing. Within the training block, the last 350 observations are used for validation.

The hyperparameters are first selected using the validation set. After that, the models are evaluated on the test set. Following the project requirement, the models are re-estimated at each forecasting step. This is done with an expanding window. At each step, the model is trained with all observations available up to that point, and then it predicts the next day. Therefore, the training sample becomes larger over time.

Forecast accuracy is measured by the Root Mean Squared Forecast Error:

$$\text{RMSFE} = \sqrt{\frac{1}{T_{\text{test}}} \sum_{t \in \text{Test}} (y_{t+1} - \hat{y}_{t+1})^2}. \quad (5)$$

A lower RMSFE means that the model gives more accurate forecasts. The models are also compared to a naive persistence benchmark, where the forecast for tomorrow is simply

today’s value:

$$\hat{y}_{t+1} = y_t$$

This benchmark has an RMSFE of 1.6953.

To compare forecast accuracy more formally, we also use Diebold-Mariano tests based on squared forecast errors (Diebold and Mariano, 1995).

3.4 Feature Importance

For the models with covariates in Part (b), we also study which variables are most important. For this, permutation feature importance is used (Molnar, 2025). The idea is simple: one variable is randomly shuffled, while all other variables stay unchanged. If the model becomes much worse after shuffling this variable, then the variable is important for the forecast.

For each variable, the increase in RMSFE after shuffling is used as the importance score. Each variable is shuffled ten times, and the average increase in RMSFE is reported. This makes the importance values easier to compare with the main forecast results.

3.5 Software

The analysis is done in Python using scikit-learn (Pedregosa et al., 2011). The regression tree is estimated with `DecisionTreeRegressor`, and the neural network is estimated with `MLPRegressor`. The MLP input variables are standardised with `StandardScaler`. Permutation importance is computed using scikit-learn’s `permutation_importance` function.

4 Empirical Results

4.1 Part (a): Forecasting with Lags Only

In Part (a), the models only use the ten most recent lags of the log case series. For the regression tree, the best validation result is obtained with a maximum depth of 7 and a minimum leaf size of 20. For the MLP, the best architecture uses two hidden layers with 128 and 64 units. The test-set RMSFE values are shown in Table 1.

Table 1: Out-of-sample test-set RMSFE for all model configurations.

Model	Feature configuration	Test RMSFE
Naive persistence	–	1.6953
Regression tree (CART)	Part (a): lags only	0.7107
MLP	Part (a): lags only	0.5951
Regression tree (CART)	Part (b): lags and covariates	0.7252
MLP	Part (b): lags and covariates	0.5715

Both machine learning models perform much better than the naive persistence benchmark. The MLP gives the best result in Part (a), with an RMSFE of 0.5951. The regression tree has a higher RMSFE of 0.7107. The Diebold-Mariano test also shows that this difference is statistically significant ($DM = 2.949$, $p = 0.003$).

This means that the MLP is better able to use the information from the lagged case values. One possible reason is that the MLP can combine all lagged variables in a smoother way, while the regression tree uses fixed split points. The tree is easier to interpret, but it may be less flexible for this kind of time series.

When inspecting the fitted tree, the first split is made on `log_cases_lag6`. This happens in both Part (a) and Part (b). This suggests that the tree finds the weekly reporting pattern in the data, because the 6-day lag is close to a weekly cycle.

Figure 1 shows the forecasts from Part (a) together with the actual log case numbers.

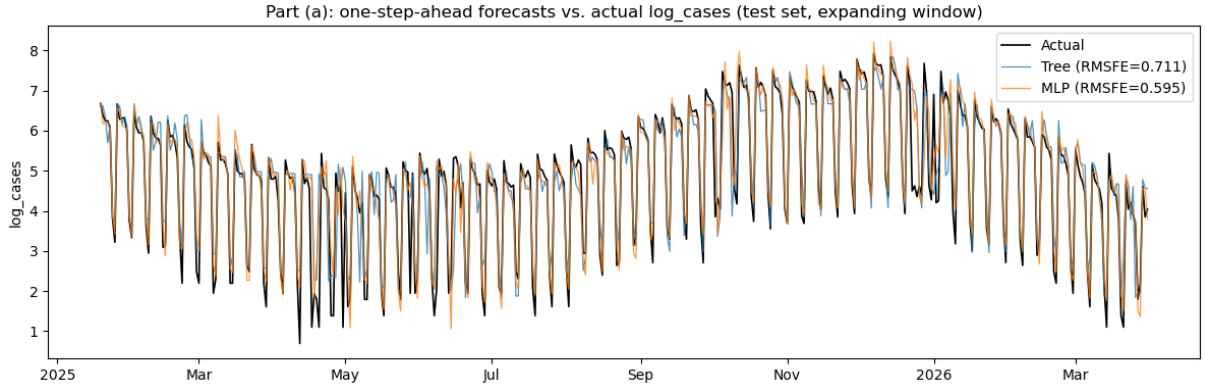


Figure 1: One-step-ahead forecasts and actual log-transformed cases on the test set, using only lagged cases.

4.2 Part (b): Forecasting with Lags and Covariates

In Part (b), the feature set is larger because lagged covariates are added. The regression tree now has a maximum depth of 5 and a minimum leaf size of 10. Its RMSFE increases slightly from 0.7107 to 0.7252. In contrast, the MLP improves from 0.5951 to 0.5715. The best MLP architecture in this part has two hidden layers with 64 and 32 units.

The Diebold-Mariano tests show that the changes from Part (a) to Part (b) are not statistically significant. For the tree, the test gives $DM = -0.968$ and $p = 0.333$. For the MLP, the test gives $DM = 0.842$ and $p = 0.3996$. However, the MLP still performs significantly better than the regression tree in Part (b), with $DM = 3.325$ and $p = 0.0009$. The results are shown in Table 2.

Figure 2 shows the forecasts from Part (b). The MLP follows the actual series more closely than the regression tree. This is also visible in the lower RMSFE value.

Table 2: Diebold-Mariano test results using squared forecast errors.

Method 1	Method 2	DM statistic	p -value	Result
Tree (a)	MLP (a)	2.949	0.003	Significant
Tree (b)	MLP (b)	3.325	0.0009	Significant
Tree (a)	Tree (b)	-0.968	0.333	Not significant
MLP (a)	MLP (b)	0.842	0.3996	Not significant

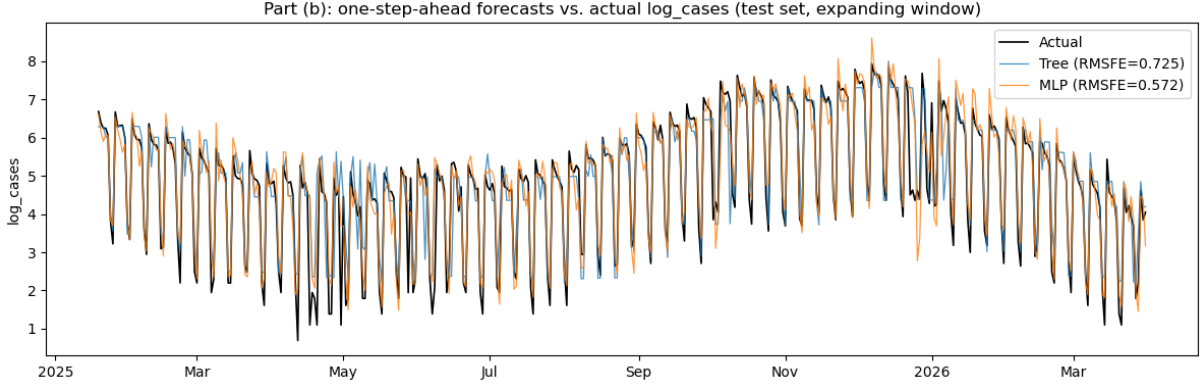


Figure 2: One-step-ahead forecasts and actual log-transformed cases on the test set, using lagged cases and covariates.

The regression tree becomes slightly worse after adding the covariates. One reason may be that the tree has many more possible variables to choose from, but it is still quite shallow. Therefore, it may choose splits that work well in the training data but do not improve the test forecasts. Figure 3 gives a supporting check for this behaviour. It shows that the training error keeps decreasing as the tree becomes deeper, while the validation and test errors do not improve in the same way. This indicates that deeper trees are more likely to overfit.

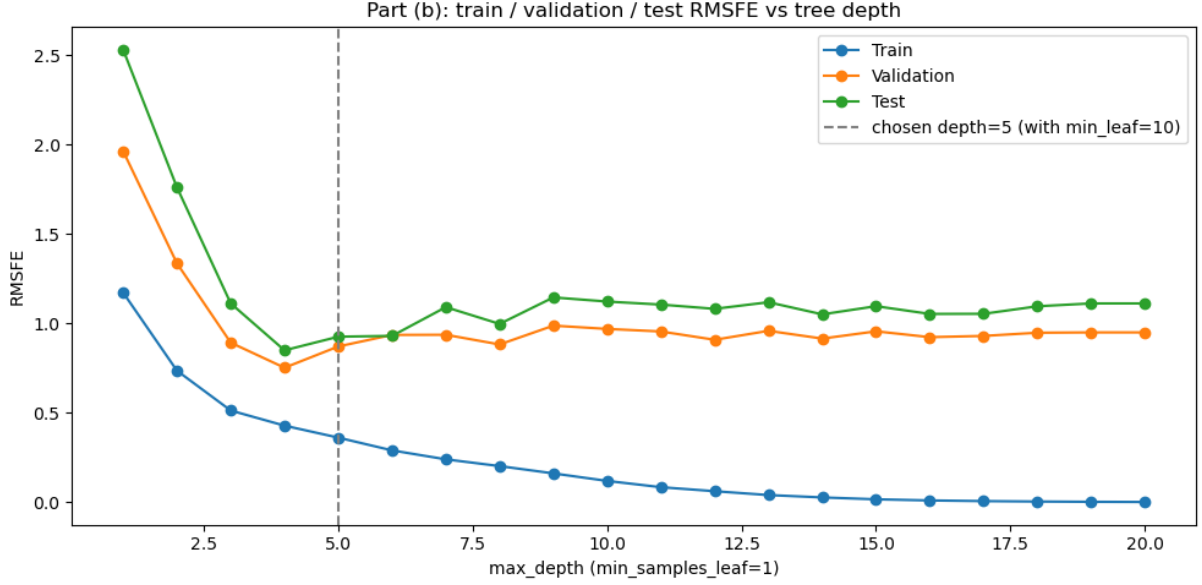


Figure 3: Training, validation, and test RMSFE of the regression tree for different maximum depths in Part (b).

For the MLP, adding covariates improves the RMSFE. Although this improvement is not statistically significant, it suggests that the neural network can use some of the additional information, especially the calendar and age-group variables.

4.3 Parts (c) and (d): Variable Importance

Table 3 shows the ten most important variables for both models in Part (b). The importance value is the increase in RMSFE after randomly shuffling the variable. A larger value means that the variable is more important for the forecast.

Table 3: Top 10 features by permutation importance in Part (b).

Regression tree (CART)			MLP		
Rank	Feature	Δ RMSFE	Rank	Feature	Δ RMSFE
1	log cases lag 6	1.498	1	start of week lag 0	0.530
2	log cases A60–A79 lag 2	0.091	2	start of week lag 6	0.444
3	start of week lag 5	0.034	3	start of week lag 3	0.391
4	log cases A15–A34 lag 5	0.034	4	start of week lag 5	0.333
5	log cases A80plus lag 0	0.034	5	start of week lag 4	0.295
6	log cases lag 0	0.021	6	log cases A80plus lag 0	0.278
7	log cases A15–A34 lag 0	0.016	7	log cases A80plus lag 4	0.243
8	log cases A35–A59 lag 0	0.001	8	start of week lag 9	0.207
9	log cases A60–A79 lag 6	0.000	9	start of week lag 8	0.188
10	log cases A80plus lag 6	0.000	10	start of week lag 7	0.167

Figure 4 gives the same information as a plot. The results show that the two models use the variables in different ways.

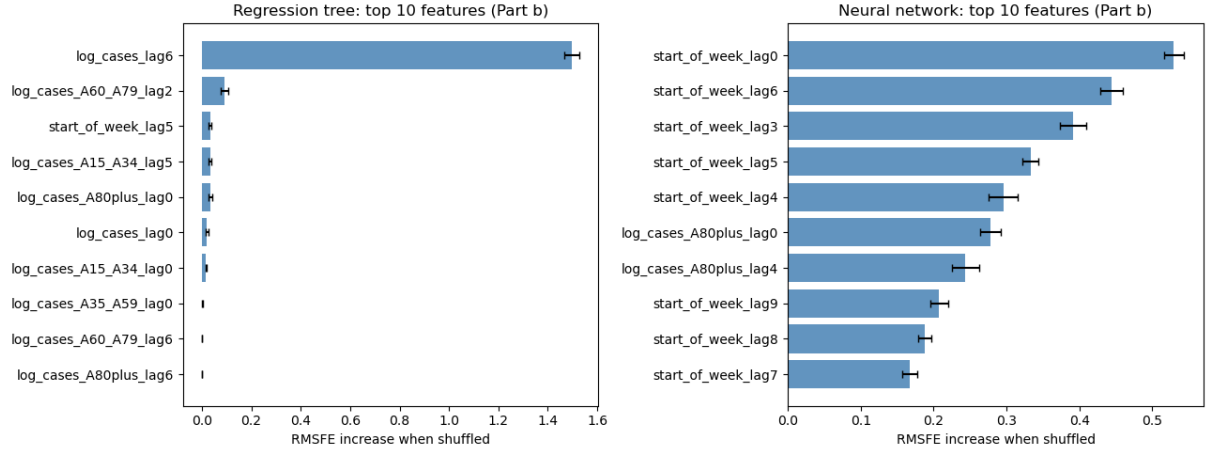


Figure 4: Permutation feature importance for the regression tree and the MLP in Part (b).

For the regression tree, the most important variable is clearly `log_cases_lag6`. Its importance value is much larger than all other variables. This shows that the tree mainly depends on one lag of the case series. Most other variables have very small importance values.

For the MLP, the importance is more spread out. Most of the important variables are lags of the Monday dummy `start_of_week`. The most important one is start of week lag 0, followed by several other lags of the same weekly indicator. This means that the MLP uses the weekly reporting pattern more directly than the regression tree. Age-group variables also appear among the important features, especially `log_cases_A80plus`. This suggests that age-specific case numbers contain useful information for the forecasts.

The weekday pattern is also visible in the residuals. Figure 5 shows the mean forecast residual for each weekday. The regression tree overpredicts especially on Saturdays, with a mean residual of -0.458 . The MLP also has a negative Saturday residual, but it is smaller at -0.202 . This means that the MLP handles the weekly pattern better than the tree, although it does not remove the bias completely.

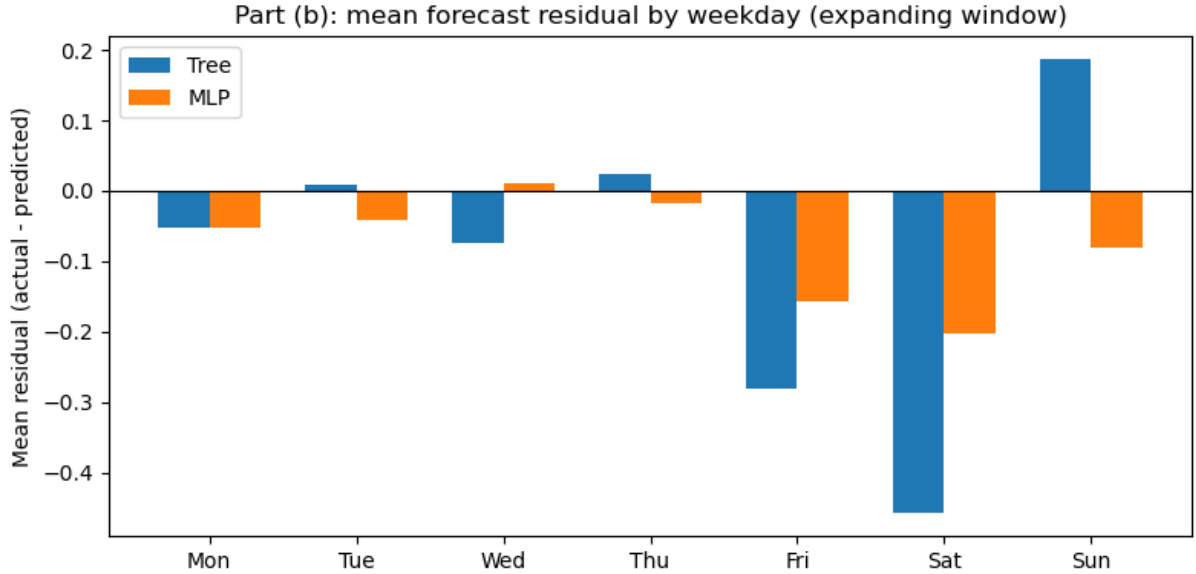


Figure 5: Mean forecast residual by weekday in Part (b).

One limitation of the importance results is that several lags of the Monday dummy contain similar information. Because of this, the importance of the weekly pattern is split across several variables. Therefore, the total importance of the calendar effect may be larger than the value of any single dummy lag.

5 Conclusion

This report compared a regression tree and a multi-layer perceptron for one-step-ahead forecasting of daily log-transformed COVID-19 cases in Germany. The models use a maximum lag length of ten days and are evaluated with an expanding-window forecasting setup. The data comes from the RKI case data from 2020 to 2026.

The results show that the MLP performs better than the regression tree in both model settings. In Part (a), where only lagged case values are used, the MLP has an RMSFE of 0.5951, while the regression tree has an RMSFE of 0.7107. The Diebold-Mariano test shows that this difference is statistically significant ($DM = 2.949$, $p = 0.003$). In Part (b), where covariates are added, the MLP improves to an RMSFE of 0.5715, while the regression tree becomes slightly worse with an RMSFE of 0.7252. The MLP also performs significantly better than the tree in this setting ($DM = 3.325$, $p = 0.0009$).

Adding covariates does not lead to a statistically significant improvement for either model. However, the direction of the results is different. The MLP becomes better when more variables are added, while the regression tree becomes slightly worse. This suggests that the MLP can use the additional information more effectively than the tree.

The variable importance results also show clear differences between the two models. The regression tree mainly depends on one variable, `log_cases_lag6`. This means that the tree mostly uses one lag of the case series for prediction. The MLP uses a wider set of variables, especially lags of the Monday dummy and age-group case numbers. This shows that the MLP captures the weekly reporting pattern more directly. The residual analysis also supports this result, because the tree has a stronger Saturday overprediction bias than the MLP.

There are some limitations to this analysis. First, only a single regression tree is used. Other tree-based methods, such as Random Forests ([Breiman, 2001](#)), XGBoost ([Chen and Guestrin, 2016](#)), or LightGBM ([Ke et al., 2017](#)), could perform better. Second, the expanding-window method requires many model refits, which can become computationally expensive for larger datasets. Third, permutation importance shows which variables are useful, but it does not fully explain how the neural network combines them. Finally, the models assume that the past reporting patterns remain useful for the future. If testing rules or reporting systems change again, the forecasts may become less reliable.

Future work could extend the analysis in several ways. First, ensemble tree methods such as Random Forests or boosting could be included, since they usually perform better than a single regression tree. Second, the lag window could be increased beyond ten days to check whether longer weekly or monthly patterns improve the forecasts. Third, other covariates could be added, for example testing intensity, vaccination rates, hospitalisation numbers, or public holiday indicators, if these data are available. It would also be useful to compare the machine learning models with classical time-series models from Project 1 in the same test period. Finally, future work could test whether a combined model works better, for example by modelling the trend and seasonality with a time-series model and then using machine learning for the remaining residual patterns.

Bibliography

- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer, New York.
- Box, G. E. P. and Cox, D. R. (1964). An analysis of transformations. *Journal of the Royal Statistical Society: Series B*, 26(2):211–252.
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1):5–32.
- Breiman, L., Friedman, J. H., Olshen, R. A., and Stone, C. J. (1984). *Classification and Regression Trees*. Wadsworth, Belmont, CA.
- Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794. ACM.
- Diebold, F. X. and Mariano, R. S. (1995). Comparing predictive accuracy. *Journal of Business & Economic Statistics*, 13(3):253–263.
- Györfi, L., Kohler, M., Krzyżak, A., and Walk, H. (2002). *A Distribution-Free Theory of Nonparametric Regression*. Springer, New York.
- Hastie, T., Tibshirani, R., and Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, New York, 2 edition.
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30*, pages 3149–3157.
- Kucharski, A. J., Russell, T. W., Diamond, C., Liu, Y., Edmunds, J., Funk, S., and Eggo, R. M. (2020). Early dynamics of transmission and control of COVID-19: A mathematical modelling study. *The Lancet Infectious Diseases*, 20(5):553–558.
- Merow, C. and Urban, M. C. (2020). Seasonality and uncertainty in global COVID-19 growth rates. *Proceedings of the National Academy of Sciences*, 117(44):27456–27464.
- Molnar, C. (2025). *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*. 3 edition. Accessed: 16 June 2026.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Mueller, A., Nothman, J., Louppe, G., Prettenhofer, P., Weiss, R., Dubourg, V., VanderPlas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, É. (2011). Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830.
- Robert Koch-Institut (2026). SARS-CoV-2 infektionen in deutschland. <https://github.com>.

- [com/robert-koch-institut/SARS-CoV-2-Infektionen_in_Deutschland](https://open-meteo.com/robert-koch-institut/SARS-CoV-2-Infektionen_in_Deutschland). Open-data repository and documentation; accessed: 16 June 2026.
- Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088):533–536.
- Tashman, L. J. (2000). Out-of-sample tests of forecasting accuracy: An analysis and review. *International Journal of Forecasting*, 16(4):437–450.
- Zippenfenig, P. (2026). Open-Meteo.com weather api. <https://open-meteo.com/>. Historical Weather API; DOI: 10.5281/zenodo.7970649; accessed: 16 June 2026.